Revision 46-49 · Prefix Caching

Interactive revision of lesson 46, lesson 47, lesson 48 and lesson 49: each question as asked, the answer, where I slipped, and the one-line takeaway.

Q1 · The request ID on line 2 PARTIAL

Covers: 11, 46

> **QUESTION**
>
> A support assistant sends ~3,300 prompt tokens per request, ~2,900 of them identical across users, but the measured prefix hit rate is ~1%. The prompt builder puts request_id and the current time on line 2. Why can't the identical tokens after it be reused, what's the fix, and how much prefill does it save at ~32 μs per prompt token with ~400 truly new tokens?

How I solved it

1. The request ID and time change on every request, so the block holding them never matches.
2. Every token's K and V depend on **all tokens before it** (causal attention), so once one token differs, every later token's K and V differ too, even if the text is identical.
3. vLLM enforces this with **chained block hashes**: block k's hash includes block $k-1$'s hash, so a miss in block 2 makes every later block miss.
4. Fix: move the request ID and time to the **end** of the prompt (or out of it).
5. Prefill before = $3{,}300 \times 32\,\mu s \approx$ **105 ms**; after = $400 \times 32\,\mu s \approx$ **13 ms**. Saved ≈ **92 ms per request**, about **8×** less prefill.

> **RE-DRILL**
>
> When asked "why not reused", explain the tokens after the change, not the change itself.

Q2 · Ideal 75%, measured 30% MISSED

Covers: 46, 48

> **QUESTION**
>
> Replaying an hour of real prompts with the server's chained 16-token block hashing gives an ideal hit rate of 75%. vLLM's prefix_cache_hits ÷ prefix_cache_queries over the same hour gives 30%. What causes the gap, and if the replay were low too, what plot finds the line to fix?

Answer

- The replay is an **unlimited cache** on the prompts as sent. A high ideal with a low measured rate means the prompts are fine and the **server** loses the reuse:

- **Cache too small:** blocks are evicted before they come back. Check the eviction count and KV usage; give the cache more memory (utilization, fp8 KV) or reduce churn.
- **A setting the replay didn't copy:** a different chat template or tokenizer, a per-tenant `cache_salt`, or **several replicas behind a random load balancer**, so each replica only sees a slice of the repeats (fix: route by prefix).
- If the **replay is low too**, the prompts break the prefix. Build a **first-divergence histogram**: compare each request with the previous one from the same app, record the first token position where they differ, and plot the counts. Healthy traffic diverges late, near the user message; a **tall spike at a low position** (e.g. token 30) points at the dynamic line, like a request ID.

> **RE-DRILL**
>
> Re-drill: name three things that make measured < ideal without any prompt change.

Q3 · 80% hits, but not 80% faster PARTIAL

Covers: 03, 16, 47

> **QUESTION**
>
> With an 80% prefix hit rate, the PM expected TTFT to drop 80% and responses to get ~5× faster. Instead mean TTFT went 108 → 68 ms, end-to-end time for long answers (550-token prompts, 1,000-token replies) barely moved, and a nightly batch job with 49% hits got 1.82× input throughput. Explain all three, and give the batch job's ceiling.

Answer

- **TTFT fell, but only 37%:** hits skip prefill for the cached part, but part of TTFT doesn't shrink with fewer prompt tokens: scheduling and queueing, the uncached tail (including the partial last block), and sampling the first token.
- **Long answers barely moved:** prefix caching only removes prefill. Each of the 1,000 decode steps still reads the weights and the full KV cache, so decode speed is unchanged (TPOT 29 → 25 ms only because less prefill competes for the GPU).
- **Batch job 1.82×:** a batch job is almost all prefill, so skipping 49% of prompt tokens nearly removes 49% of the work. Ceiling (Amdahl) = $1 \div (1 - 0.49) \approx$ **1.96×**; 1.82× is close to it.

> **RE-DRILL**
>
> Re-drill: whenever a fraction of work is removed, write Amdahl's $1 \div (1 - f)$ before predicting a speedup.

Q4 · Full blocks and tenant salts SOLID

Covers: 38, 47

> **QUESTION**
>
> Customers A and B use the same 1,500-token system prompt. With 16-token blocks, how many of those tokens can a later request reuse, and why not all? Compliance says A and B must never share cached K/V; how do you enforce that without turning prefix caching off?

How I solved it

1. 1,500 ÷ 16 = 93.75, so **93 full blocks = 1,488 tokens** are reusable.
2. The last 12 system-prompt tokens share block 94 with the start of the user's message, which differs per request, so that block misses and gets recomputed. Only full, identical blocks are cached.
3. Give each tenant a `cache_salt`: it's mixed into the block hashes, so identical prompts from A and B produce different fingerprints and never match, while each tenant still reuses its own prefix.

> **RE-DRILL**
>
> The salt only needs to enter the first block's hash: the chain carries it into every later block.

Q5 · Six turns, then trimming PARTIAL

Covers: 14, 48

> **QUESTION**
>
> A chat has a 1,000-token system prompt; each turn adds a 100-token question and a 300-token answer. Over 6 turns, how many prefill tokens without reuse and with a perfect prefix cache? At turn 7 the app starts trimming the oldest message every turn; what does that do to reuse, and how would you trim instead?

How I solved it

1. Without reuse, turn n re-sends everything: 1,000 + 100n + 300(n − 1) tokens, i.e. 1,100, 1,500, 1,900, 2,300, 2,700, 3,100. Total **12,600 tokens**, growing with the square of the turns.
2. With reuse, each turn finds the whole previous conversation cached, **including the model's own answer** (its K and V were computed while generating). Each turn computes only the new question plus the 8 leftover tokens of the last partial block: 108 tokens.
3. Total = 1,100 + 5 × 108 = **1,640 tokens** (the lesson rounds to ≈ 1,700), about **7×** less and growing linearly.
4. Trimming the oldest message changes the tokens right after the system prompt, so everything after the system prompt's 62 full blocks misses: turn 7 recomputes **~2,108 tokens instead of 108**, and again every turn.
5. Better: keep history **append-only** as long as possible; when you must cut, keep the start intact and cut a **big chunk rarely** (or summarize once), so the recompute is paid once instead of every turn.

RE-DRILL

Re-drill: compute the per-turn leftover (conversation length mod 16) and turn 20's gap (31×).

Q6 · RAG order and replica routing SOLID

Covers: 46, 48

QUESTION

A RAG prompt is 800 tokens of instructions, 5 retrieved passages × 300 tokens, then a ~50-token question. Two users get the same passages in a different rank order. What share of user 2's prompt is reused, and what pushes it near 100%? On 4 replicas behind a round-robin load balancer, what happens to the hit rate, and what's the fix?

How I solved it

1. Matching runs from the start: the instructions match, then passage 1 vs passage 3 differ and matching stops. Reused ≈ 800 ÷ 2,350 ≈ **34%**.
2. Sort passages into a **fixed order** (by document ID, not score): instructions plus all five passages line up, (800 + 1,500) ÷ 2,350 ≈ **98%**. Only the question is new.
3. Round-robin sends each request to a replica that probably hasn't seen its prefix: the cache lives on one GPU, so a repeat finds it with probability about **1/4**.
4. Fix: **route by a hash of the prompt's first tokens** (what OpenAI's API does), so requests sharing a prefix land on the same replica.

RE-DRILL

Re-drill: for any routing key, ask "do two requests that share a prefix get the same key?"

Q7 · 47 shared tokens, and who gets evicted PARTIAL

Covers: 40, 47, 49

QUESTION

Two prompts share their first 47 tokens. How many does vLLM's prefix cache reuse, and SGLang's RadixAttention, and why? When memory runs out, which radix-tree nodes can SGLang never evict, which go first, and why does the shared system prompt survive longest?

How I solved it

1. vLLM caches full 16-token blocks: ⌊47 ÷ 16⌋ = 2 blocks = **32 tokens** reused; the other 15 are recomputed.
2. SGLang's radix tree matches **token by token**, splitting an edge exactly where prompts differ: all **47 tokens** reused.

3. **Never evicted:** nodes on the path of a running request; each running request **pins** its path with a reference count, so only refcount-0 nodes are candidates.
4. **First to go:** unpinned **leaves**, the **least recently used** first.
5. A parent only becomes a leaf once all its children are gone, so a node many requests pass through (the system prompt) is the last to become evictable: **shared trunks survive longest**.

> **RE-DRILL**
>
> Re-drill: state the two filters in order: pinned? then leaf? then oldest.

Q8 · Churn and cache-aware order PARTIAL

Covers: 36, 49

> **QUESTION**
>
> The cache holds 4 prefixes; traffic cycles through 5: A, B, C, D, E, A, B, C, D, E. Under LRU, how many of the 10 requests hit, and why does memory for one more prefix make such a jump? If the scheduler serves the longest cached prefix first, why does the hit rate rise, and what's the risk?

Answer

- **0 of 10 hit.** E evicts A (the least recently used); then A returns and evicts B, B evicts C, and so on: LRU always throws out exactly the prefix needed next.
- With room for **5**, the whole working set fits: the second cycle hits every time (5 of 10 here, 100% in steady state). Under cyclic traffic, LRU's hit rate is **0% until the cache holds the whole working set, then jumps to 100%**: a cliff, not a slope.
- **Longest cached prefix first** serves every waiting request that shares a cached prefix **before** that prefix is evicted. That walks the radix tree **depth-first**, finishing one branch before the next, so the cache stops switching between groups. The SGLang paper proves it gives the best hit rate for a fixed batch (when the cache holds at least the longest request); grouping the same 10 requests by prefix gets 5 of 10 even with room for only 4.
- **Risk: starvation.** A request with no shared prefix keeps getting pushed behind ones that do; it needs an age limit or fairness rule.
- Churn shows up as **low hit rate plus high eviction count**. Fixes: more cache memory, route by prefix, a CPU memory tier, popularity-aware eviction.

> **RE-DRILL**
>
> Re-drill: simulate a cache of 1 with arrivals ABCABCABC vs AAABBBCCC (0 vs 6 hits).